



SMT(LIA) Sampling with High Diversity^{*}

Yong Lai¹, Junjie Li², and Chuan Luo³

¹ Key Laboratory of Symbolic Computation and Knowledge Engineering Ministry of Education, Jilin University, China

² College of Software, Jilin University, Changchun, China

³ School of Software, Beihang University, Beijing, China

laiy@jlu.edu.cn, junjieli2026@gmail.com, chuanluo@buaa.edu.cn

Abstract. SMT sampling refers to the task of generating a set of satisfying assignments (samples) for a given SMT formula. An effective SMT sampler should be capable of producing samples with high diversity to maximize coverage of the solution space. However, most SMT samplers struggle to adequately cover the solution space and fail to generate sufficiently diverse solutions.

To address these limitations, we propose *HighDiv*, the first iterative sampling framework that integrates CDCL(T) and local search in a bidirectional guided manner. During the bidirectional guidance process, solutions generated by CDCL(T) guide the variable initialization of the local search. Conversely, solutions produced by the local search guide CDCL(T) to further explore the solution space. Additionally, we design a novel local search algorithm, *Context-Constrained Stochastic Search (CCSS)*, which introduces the *Constraint-Partitioned Variable Initialization* and the *boundary-aware move* operator. These components effectively balance exploration and feasibility throughout the search process. We conduct an extensive evaluation on QF_LIA formulas from the SMT-LIB benchmark. The results demonstrate that *HighDiv* achieves substantial improvements in diversity over the state-of-the-art SMT sampling tools.

1 Introduction

Satisfiability Modulo Theories (SMT) is the problem of deciding the satisfiability of a first-order logic formula with respect to certain background theories. Modern software/hardware testing widely leverages SMT solvers to generate assignments that satisfy specified constraints, which are then used to construct test suites. In software testing, symbolic execution employs SMT solvers to solve collected path constraints, thereby producing new inputs that cover unexplored paths [4, 40, 42]; in hardware testing, constrained-random verification (CRV) encodes the functional model and verification scenarios of a hardware design as an SMT formula, whose solutions correspond to stimuli for the design under test (DUT) [38].

^{*} The author list has been sorted alphabetically by last name.

Although SMT solving has been extensively studied and mainstream solvers typically return a single solution within acceptable time, software/hardware testing often requires multiple solutions. In software testing, prior work has shown that obtaining multiple solutions for the same constraint during symbolic execution can substantially improve testing effectiveness [19, 20, 28, 48]. In hardware testing, within CRV, diversity among solutions directly translates into richer stimuli, thereby increasing the likelihood of exposing bugs [38].

The task of generating a set of highly diverse solutions for a given SMT formula is known as the sampling problem, which has attracted more attention in the community in recent years [7, 10, 11, 39]. Although using a solver to enumerate solutions (by adding blocking constraints) for a formula is operationally straightforward, this approach is generally costly, and after multiple invocations, most solvers tend to return similar solutions. This prevalence of similar solutions is largely attributed to the fact that many solver techniques (e.g., the general simplex method for linear theories) tend to favor extreme values at the boundaries of the feasible region [1, 8, 36]. Therefore, there is a practical need to efficiently generate a highly diverse set of solutions for SMT formulas.

Existing SMT samplers typically treat SMT solvers as black boxes [10, 11, 39]. Their core strategy involves obtaining solutions from SMT solvers as initial seeds, which are then used to generate additional solutions. However, this approach inherently limits the diversity of generated samples as a large number of samples originate from a very limited set of seeds.

Recent studies on Boolean satisfiability (SAT) sampling have demonstrated that treating the solving procedure as a white-box and deliberately introducing diversification during search can substantially improve both effectiveness and efficiency [17, 30, 31]. In the context of finding a single solution of SMT problems, Conflict-Driven Clause Learning with Theory (CDCL(T)) and local search are two widely recognized and effective algorithmic frameworks. These two approaches are highly complementary, as demonstrated by their combined use in various recent studies [5, 47]. Specifically, CDCL(T) leverages its powerful CDCL engine to perform efficient reasoning at the propositional logic level, while local search achieves rapid solving by flexibly modifying the current assignment. However, existing SMT solvers focus only on quickly finding a satisfying assignment without considering solution diversity during the assignment generation process. Consequently, striking an appropriate balance between sample diversity and search efficiency remains a challenge in SMT sampling tasks. This raises a fundamental question: *Can we combine CDCL(T) with local search in a white-box way to design high-diversity SMT sampling?*

In this paper, we introduce *HighDiv*, a novel sampling framework for the theory of *linear integer arithmetic* (SMT(LIA)) that is designed to tackle the aforementioned challenge in SMT sampling. We focus on SMT(LIA) because it is widely used in software and hardware testing and verification [9, 15, 29, 32]. The framework utilizes bidirectional guidance between local search and CDCL(T) for iterative sampling, aiming to enhance sample diversity while maintaining solving efficiency. Specifically, the solution from CDCL(T) guides variable initialization

during the local search phase, while the solution generated by local search is fed back into CDCL(T) to further explore the solution space.

In addition, for the local search component, we designed *Context-Constrained Stochastic Search (CCSS)*, which introduces a novel variable initialization strategy called the *Constraint-Partitioned Variable Initialization*. This strategy partitions the constraints into different subsystems and initializes each subsystem based on its specific characteristics. Furthermore, to allow more flexible modifications of integer variables during the search, we introduced a new operator, *boundary-aware move*. The operator performs random moves within the interval determined by contextual boundary information. For the CDCL(T) component, we modified its *branching heuristic* and *phase selection heuristic*, enabling more flexible exploration of the solution space at the propositional level.

Under a 900-second time limit, *HighDiv* covers approximately 53% of the SMT-formula AST space, whereas state-of-the-art SMT(LIA) samplers average about 34%. Under a fixed sampling budget of 1000 samples, *HighDiv* reaches roughly 53% coverage, while competitors remain below 35%. These results clearly indicate that *HighDiv* is significantly more diverse than state-of-the-art SMT(LIA) samplers.

The primary contributions of this work are outlined below.

- We propose *HighDiv*, a SMT sampling framework that combines CDCL(T) and local search with bidirectional guided iteration, and treats the search process as a white-box, thereby advancing SMT(LIA) sampling.
- We investigate the trade-off between sample diversity and search efficiency in our local search algorithm, Context-Constrained Stochastic Search (*CCSS*), featuring a novel variable initialization strategy and the *boundary-aware move* operator.
- We implement *HighDiv* and compare it against state-of-the-art SMT(LIA) samplers. The results demonstrate that *HighDiv* generates more diverse samples under both fixed-time and fixed-sample-size conditions.

The structure of this paper is as follows: Section 2 introduces the relevant definitions and the frameworks of local search and CDCL(T); Section 3 presents the overall framework of *HighDiv*, as well as the interaction between stochastic CDCL(T) and *CCSS*; Section 4 introduces the stochastic CDCL(T) component; Section 5 provides a detailed explanation of *CCSS*; Section 6 presents the experiments; Section 7 discusses related work; Section 8 concludes.

2 Preliminary

2.1 SMT(LIA)

A formula under the Linear Integer Arithmetic (LIA) theory is constructed from *atomic formulas* using Boolean connectives, where an atomic formula is either a Boolean variable or an arithmetic formula. Arithmetic formulas can be consistently expressed in the standard form $\sum_{i=0}^{n-1} a_i x_i \leq k$ or $\sum_{i=0}^{n-1} a_i x_i = k$, where x

represents an integer variable, with a and k as constants. A *literal* can be either an atomic formula or its negated form. A *clause* is composed of the disjunction of a set of literals, and a formula in *Conjunctive Normal Form* (CNF) consists of the conjunction of a set of such clauses. Given sets of propositional variables $\mathcal{P}$ and integer variables $\mathcal{X}$, which are integral components of the SMT(LIA) formula F , an *assignment* α of F maps each $x \in \mathcal{X}$ to an integer in $\mathbb{Z}$ and each $p \in \mathcal{P}$ to *true* or *false*. Under this assignment, $\alpha(x)$ and $\alpha(p)$ represent the values of x and p , respectively.

Example 1. Given a set of integer variables $\mathcal{X} = \{x_1, x_2, x_3, x_4\}$ and a set of Boolean variables $\mathcal{P} = \{p_1, p_2\}$, the following F is a SMT(LIA) CNF formula,

$$\begin{aligned} F = & (p_1 \vee \neg p_2) \\ & \wedge (\neg(x_1 + x_2 \leq 2) \vee (-2x_1 + 3x_3 \leq 0)) \\ & \wedge (p_2 \vee (3x_2 - 7x_3 \leq 3)). \end{aligned}$$

2.2 SMT Sampling

Let φ be an SMT formula and $\text{Sol}(\varphi)$ denote the set of all its satisfying assignments. The *SMT sampling problem* aims to construct, under limited resources (e.g., a sample-size bound or a time limit), a solution set $\mathcal{S} \subseteq \text{Sol}(\varphi)$. Unlike conventional SMT solving, which returns an arbitrary satisfying assignment, sampling focuses on the diversity of solutions.

To measure diversity, we adopt the abstract syntax tree (AST) coverage metric proposed in prior work [10]. Specifically, we consider nodes in the AST of the SMT formula. Each Boolean node is treated as a single bit, while for integer nodes, only the least significant 64 bits of their values are considered. A bit is regarded as covered if it takes both values 0 and 1 across the sample set of satisfying models [39]. The overall coverage is defined as the ratio between the number of covered bits and the total number of bits. Note that under this metric, the maximum achievable coverage is not necessarily 100%, since some AST node values may be constrained by the input formula and thus take the same value for all assignments in $\text{Sol}(\varphi)$.

2.3 CDCL(T) Framework

Most modern SMT solvers—e.g., Z3 [36] and CVC5 [1]—solve SMT(LIA) formulas using the CDCL(T) framework: a CDCL SAT engine reasons over the formula’s Boolean skeleton while a LIA theory solver checks consistency, performs theory propagation, and returns theory lemmas to guide the SAT search [3, 14, 23].

The efficiency of CDCL critically depends on two heuristics: (i) *branching heuristics*, which choose the next decision variable (e.g., VSIDS [35]), and (ii) *phase selection heuristics*, which decide the sign of that variable (e.g., phase saving [41]). Once a complete assignment satisfying the formula is found, CDCL(T) returns the current assignment α , which serves as a *model* of the formula. More details about the CDCL(T) algorithm can be found in the references [3, 14, 23].

2.4 Local Search Framework for SMT(LIA)

The local search component of *HighDiv* uses the two-mode framework of LS-LIA, the first local search algorithm for SMT(LIA) [5]. We briefly introduce LS-LIA.

This algorithm divides its search process into two modes: *Boolean mode* and *Integer mode*. In different modes, operations on variables of the appropriate data type are selected to modify the current assignment. In each mode, when the number of *non-improving steps* reaches the threshold, it switches to another mode. This threshold is defined as $L \times P_b$ for the Boolean mode and $L \times P_i$ for the Integer mode, where P_b and P_i represent the proportion of Boolean and Integer literals, respectively, in the falsified clauses, and L is a parameter.

Within a local search algorithm, the pivotal element is the *operator*, which prescribes the manner in which the current solution may be altered. Once the operator is concretized by selecting a particular variable and designating a value to assign, it materializes as a specific *operation*. In Boolean mode, the *flip* operator turns a Boolean variable to its opposite value. The Integer mode introduces a unique operator called the *critical move* (cm), which is defined below.

Definition 1. *The critical move operator, represented as $cm(x, \ell)$, sets the integer variable x to a threshold value that satisfies the literal ℓ , where ℓ is a falsified literal that includes x .*

The threshold mentioned above refers to the minimal modification required to make the literal ℓ true for x . Example 2 is provided to assist readers in understanding the definition.

Example 2. Given the SMT(LIA) formula, which includes a literal, $\ell : (x_1 - 5x_2 \leq -5)$, the initial variable assignments are $\{x_1 = 0, x_2 = 0\}$. The operations $cm(x_1, \ell)$ and $cm(x_2, \ell)$ involve assigning -5 to x_1 and 1 to x_2 , respectively, making the literal ℓ true.

Whenever a $cm(x, \ell)$ operation is performed, the corresponding literal ℓ is set to *true*. Therefore, during the algorithm search process, falsified literals are consistently selected and a *cm* operation is applied to make ℓ true.

For an assignment α , its *cost* is the number of clauses falsified under α ; when clause weighting is enabled, the *cost* equals the total weight of all falsified clauses [6, 46].

During local search, the *score* of a candidate operation op is usually defined as

$$score(op) = cost(\alpha) - cost(\alpha'),$$

where α' is the assignment obtained by applying op to the current assignment α . If $score(op) > 0$, the operation op is called a *decreasing* move.

3 The Framework of *HighDiv*

We combine CDCL(T) and local search in a white-box way to design a high-diversity SMT(LIA) sampling method called *HighDiv*. Figure 1 provides an

overview of the general structure and sequence of operations within *HighDiv*. The sampling framework is composed of two distinct components: stochastic CDCL(T) (Section 4) and Context-Constrained Stochastic Search (*CCSS*), a novel local search algorithm for sampling SMT(LIA) formulas and achieving high diversity (Section 5). The two components interact through bidirectional guidance: the solutions generated by *CCSS* dynamically guide the stochastic CDCL(T) to further explore the solution space, while the solutions obtained from the stochastic CDCL(T) are used to guide the variable initialization in *CCSS*.

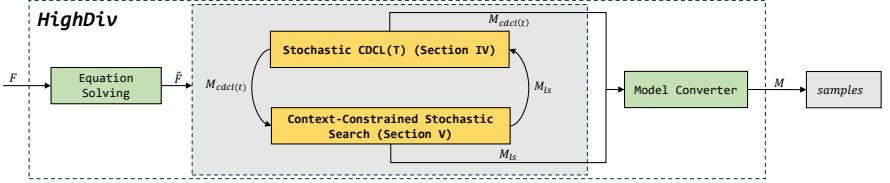


Fig. 1. Framework of *HighDiv*

Algorithm 1 presents the top-level framework of *HighDiv*. It takes two inputs: 1) the SMT(LIA) formula F , and 2) the desired number of solutions k .

HighDiv alternates between a CDCL(T) phase (lines 5-10) and a *CCSS* phase (lines 11-15). First, equalities are simplified using Z3's *solve-eqs* tactic [36], resulting in the pre-processed formula $\hat{F}$ (line 2). Next, a stochastic CDCL(T) solver attempts to solve the current under-approximation $\hat{F}_{under}$. If successful, the resulting model $M_{cdcl(t)}$ is mapped back to the original formula F and added to $\mathcal{S}$ (lines 7-8). Guided by $M_{cdcl(t)}$, the algorithm then executes *CCSS* on $\hat{F}$ (line 11). Upon success, *CCSS* returns a model M_{ls} , which is likewise converted and appended to $\mathcal{S}$ (lines 13-14). To build the next under-approximation formula, each variable in $\hat{F}$ is independently selected with a probability of P and fixed to its value in M_{ls} , yielding $\hat{F}_{under}$ (line 15). These two phases iterate, enlarging $\mathcal{S}$ until the desired sample count is reached ($|\mathcal{S}| = k$) or the time limit expires.

4 The Stochastic CDCL(T) Component

Previous studies have demonstrated that CDCL(T) and local search methods complement each other effectively [5]. A primary reason is the powerful reasoning capabilities of the CDCL engine at the propositional logic level. Inspired by SAT sampling algorithms based on stochastic CDCL [17], we implemented the stochastic CDCL(T) algorithm in Z3. Specifically, we modified the *branching heuristic* and *phase selection heuristic* in Z3, randomly selecting undecided variables at each decision point and assigning them a random phase.

Algorithm 1: The Framework of *HighDiv* (described in Section 3)

Input: F : An SMT(LIA) formula;
 k : Allowed number of samples;
Output: $\mathcal{S}$: The set of samples, i.e., solutions of F ;

```

1  $\mathcal{S} \leftarrow \emptyset$ ;
2  $\hat{F} \leftarrow \text{equation\_solving}(F)$ ;
3  $\hat{F}_{\text{under}} \leftarrow \hat{F}$ ;
4 while  $|\mathcal{S}| < k$  and time limit not reached do
    // Stochastic CDCL(T)
5    $M_{\text{cdcl}(t)} \leftarrow \text{stochastic\_cdcl}(t)\_\text{solving}(\hat{F}_{\text{under}})$ ;
6   if  $M_{\text{cdcl}(t)} \neq \emptyset$  then
7      $M \leftarrow \text{model\_convert}(M_{\text{cdcl}(t)}, F)$ ;
8      $\mathcal{S} \leftarrow \mathcal{S} \cup \{M\}$ ;
9     if  $|\mathcal{S}| = k$  then
10      return  $\mathcal{S}$ ;
    // Context-Constrained Stochastic Search
11    $M_{ls} \leftarrow \text{CCSS}(\hat{F}, M_{\text{cdcl}(t)})$ ;
12   if  $M_{ls} \neq \emptyset$  then
13      $M \leftarrow \text{model\_convert}(M_{ls}, F)$ ;
14      $\mathcal{S} \leftarrow \mathcal{S} \cup \{M\}$ ;
15      $\hat{F}_{\text{under}} \leftarrow \text{fix\_partial\_assignment}(\hat{F}, M_{ls})$ ;
16 return  $\mathcal{S}$ ;
```

Stochastic CDCL(T) plays a pivotal role in *HighDiv*. On one hand, the model generated by CDCL(T) can guide the variable initialization in *CCSS*; on the other hand, the integrated theory solvers within CDCL(T), such as the general simplex method, offer a perspective on the solution space that differs from *CCSS*, thereby enabling *HighDiv* to produce more diverse solutions.

5 Context-Constrained Stochastic Search

In this section, we first introduce Context-Constrained Stochastic Search (*CCSS*); we then detail its variable-initialization strategy and the *boundary-aware move* operator, and finally demonstrate how these components jointly mediate the trade-off between sample diversity and algorithmic scalability.

CCSS uses the basic two-mode framework of LS-LIA [5], i.e., Integer mode and Boolean mode for searching. While *CCSS* retains the LS-LIA search strategy in Boolean mode, we propose new techniques described in Sections 5.1–5.2 to improve sampling diversity in Integer mode. Algorithm 2 describes the Integer mode of *CCSS* algorithm. Line 1 generates the initial assignment α using the *Constraint-Partitioned Variable Initialization*. Lines 2-13 iteratively refine α until the number of consecutive non-improving steps exceeds the threshold *MaxSteps* or the current assignment satisfies the formula F . In lines 5-6,

Algorithm 2: Integer Mode of *CCSS* (described in Section 5)

Input: F : SMT(LIA) formula;
 M : A model of F ;
Output: α : the solution of F ;

```

1 constraint_partitioned_variable_initialization( $\alpha, M$ );
2 while  $Step_{total} \leq MaxSteps$  do
3   if  $\alpha$  satisfies  $F$  then return  $\alpha$  ;
4    $\mathcal{B} \leftarrow \emptyset$ ;
5   foreach literal  $\ell$  in falsified clauses do
6      $\mathcal{B} \leftarrow \mathcal{B} \cup \{bam(\alpha, x, \ell) \mid x \text{ appears in } \ell\}$ ;
7   if  $\exists$  decreasing bam operation in  $\mathcal{B}$  then
8      $op \leftarrow$  select such an operation with the greatest score;
9   else
10    update clauses penalty weights;
11     $c \leftarrow$  a random falsified clause with integer variable;
12     $op \leftarrow$  the bam operation with the greatest score in  $c$ ;
13   $\alpha \leftarrow \alpha$  with  $op$  performed;
```

boundary-aware moves are constructed for every literal in the falsified clause and inserted into the candidate set $\mathcal{B}$. Each iteration selects from $\mathcal{B}$ the highest-scoring *boundary-aware move* that decreases the number of falsified clauses (lines 7-8). If no such decreasing *boundary-aware move* exists, the search is deemed stuck in a local optimum; the algorithm then updates the penalty weights (line 10) and picks a random falsified clause with integer literals and chooses a *boundary-aware move* operation with the greatest score (lines 11-12).

5.1 Constraint-Partitioned Variable Initialization

In local search algorithms, the quality of variable initializations significantly impacts search efficiency [13, 27]. Traditional variable initialization methods typically rely solely on boundary information provided by the formula [5]. In the context of sampling problems, when initializing variables, it is essential to consider not only the impact of the initial values on search efficiency but also the potential detrimental effect of uniform initialization on the diversity of the solutions.

When solving constraint satisfaction problems (CSP), the constraint system can typically be decomposed into multiple independent subsystems for tailored solving. We propose a similar strategy called *Constraint-Partitioned Variable Initialization* to obtain an initialized assignment in line 1 in Algorithm 2. Specifically, we partition the integer variables into three independent constraint subsystems: the *equality system*, the *high-frequency system*, and the *general system*. Then, decomposing SMT(LIA) formulas into multiple sub-constraint sys-

tems and initializing variables based on the specific properties of the constraints achieves a better balance between sample diversity and search efficiency.

Initialization of Variables in Equality System Since *HighDiv* is integrated into Z3, we leverage Z3's equality-solving preprocessing tactic, *solve-eqs* [36]. This tactic simplifies the linear equalities in the conjunction, but may leave some equalities unresolved, which are then carried into the subsequent local search phase. Because equality constraints admit a much smaller solution space than inequality constraints, a bad initial assignment can substantially degrade the efficiency of the subsequent search.

We now define the *equality system* variable set. Let $\mathcal{C}_=$ denote equality constraints, and let $\mathcal{C}_\neq$ denote all remaining constraints. For a constraint c , let $\text{vars}(c)$ denote the set of variables occurring in c . The equality system variable set $\mathcal{E}_{eq}$ is defined as the smallest set of variables satisfying:

1. $\bigcup_{c \in \mathcal{C}_=} \text{vars}(c) \subseteq \mathcal{E}_{eq}$;
2. for every $c \in \mathcal{C}_\neq$, if $\text{vars}(c) \cap \mathcal{E}_{eq} \neq \emptyset$, then $\text{vars}(c) \subseteq \mathcal{E}_{eq}$.

To improve the efficiency of subsequent search, we initialize all variables in $\mathcal{E}_{eq}$ to zero during the variable initialization phase.

Initialization of Variables in High-frequency System In *CCSS*, each operation modifies the value of variable x to satisfy a literal ℓ that contains x . However, when x appears in multiple literals, changing its value can inadvertently disrupt the satisfaction of other literals.

For clarity, we define the frequency of a variable x , denoted by $\text{freq}(x)$, as the number of literals containing x . Let $\mathcal{C}$ be the set of constraints appearing in the formula, and let $\mathcal{E}_{eq}$ denote the set of variables in the equality system. Given a frequency threshold λ , the high-frequency variable set $\mathcal{E}_{hf}$ is defined as the smallest set of variables such that:

1. $\{v \mid \text{freq}(v) > \lambda\} \setminus \mathcal{E}_{eq} \subseteq \mathcal{E}_{hf}$;
2. for every constraint $c \in \mathcal{C} \setminus \mathcal{C}_=$, if $\text{vars}(c) \cap \mathcal{E}_{hf} \neq \emptyset$, then $\text{vars}(c) \setminus \mathcal{E}_{eq} \subseteq \mathcal{E}_{hf}$.

When initializing the variables in $\mathcal{E}_{hf}$, we first compute their value intervals based on the model obtained from the CDCL(T) phase, and then uniformly sample values within these intervals. Specifically, for each variable x to be initialized in $\mathcal{E}_{hf}$, we define the set of inequalities containing x that are true under the model M obtained in the CDCL(T) phase as $\text{lit}_{true}(x)$. For each $\ell_i \in \text{lit}_{true}(x)$, we substitute the values from M to calculate a feasible interval I_i for x under the interpretation of M with respect to ℓ_i . Therefore, each ℓ_i corresponds to a feasible interval I_i . Subsequently, the feasible intervals are merged based on the logical relationships between the literals: for conjunctions of literals ℓ_i and ℓ_j , the intervals are intersected, i.e., $I(x) = I_i \cap I_j$; for disjunctions of literals ℓ_i and ℓ_j , the intervals are united, i.e., $I(x) = I_i \cup I_j$. Finally, we obtain the initialization interval $I(x)$ for x . A specific example is given in Example 3.

Example 3. Given the SMT(LIA) formula $(\ell_1 \vee \ell_2) \wedge \ell_3 \wedge \ell_4 = (x_2 - x_1 \leq -1 \vee -x_1 \leq -10) \wedge (x_2 - x_3 \leq 0) \wedge (x_1 - x_3 \leq 3)$ and the model $M := \{x_1 = 10, x_2 = 7, x_3 = 7\}$. The literals $\{\ell_1, \ell_2, \ell_4\}$, which contain x_1 and are true under the model M , are used to compute the initialization interval of x_1 as follows:

- $\ell_1 := x_1 \geq x_2 + 1$, corresponding to $I_1 := [8, +\infty)$.
- $\ell_2 := x_1 \geq 10$, corresponding to $I_2 := [10, +\infty)$.
- $\ell_4 := x_1 \leq x_3 + 3$, corresponding to $I_4 := (-\infty, 10]$.

Therefore, the initialization interval for x_1 is $I(x_1) := (I_1 \cup I_2) \cap I_4 = [8, 10]$.

Initialization of Variables in General System Apart from the cases mentioned above, other constraints are generally easier to satisfy, and their initial states have a smaller impact on subsequent searches. Therefore, for the variables in these constraints, their values can be initialized to be far from the model obtained in the previous round of sampling.

Following Example 3, if x_1 is a general variable, the initialization interval for x_1 is set to the complement of the computed interval, i.e., $(-\infty, 7] \cup [11, +\infty)$.

5.2 Boundary-Aware Move

Previous research on local search for SMT(LIA) has introduced the *critical move* (cm) operator. Given a literal ℓ containing x and currently falsified, each operation $cm(x, \ell)$ modifies the assignment of x to make ℓ satisfied [5]. However, this operator always modifies the variable x to the boundary value that satisfies ℓ , which clearly imposes a strong restriction on the possible modified values of the variable. In fact, the value of the variable x that satisfies the literal ℓ is not unique. For example, consider the formula in Example 3 with the current assignment $\alpha := \{x_1 = 0, x_2 = 0, x_3 = 0\}$. As illustrated in Figure 2, performing the operation $cm(x_1, x_2 - x_1 \leq -1)$ assigns x_1 to 1. In fact, any value within the interval $[1, +\infty]$ for x_1 will satisfy $x_2 - x_1 \leq -1$.

However, note that when x_1 is assigned a value within the interval $[4, +\infty]$, another constraint $x_1 - x_3 \leq 3$ is no longer satisfied. In SMT(LIA) sampling, such an operation of modifying the assignment must balance search efficiency and solution diversity. To achieve a better trade-off, we propose a new operator, *boundary-aware move* (bam).

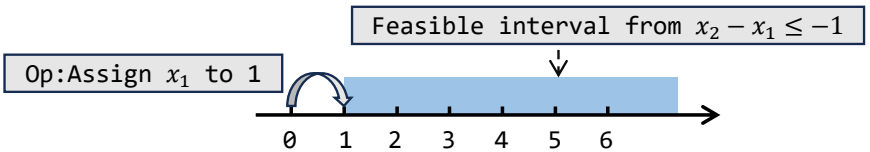


Fig. 2. When performing the operation $cm(x_1, x_2 - x_1 \leq -1)$, any assignment of the variable x_1 within $[1, +\infty]$ will make $x_2 - x_1 \leq -1$ true.

To facilitate describing bam, we first define the set of *contextual literals* for an integer variable x under assignment α as $\mathcal{L}^{\text{ctx}}(x, \alpha) = \{\ell \in \mathcal{L} \mid x \in \text{vars}(\ell) \text{ and } \alpha \models \ell\}$, where $\mathcal{L}$ is the set of all inequality literals occurring in the formula. Then, we introduce the *feasible interval* for the variable x with respect to the inequality literal ℓ : We substitute the current assignment α for every variable in ℓ except x , thereby obtaining x 's *feasible interval* I_ℓ . Under linear integer arithmetic, I_ℓ can only be $[l, \infty)$ or $(-\infty, u]$; any integer chosen from that interval guarantees that ℓ becomes true after the operation. For instance, consider the literal $\ell : (x + y \geq 5)$ and the current assignment $\alpha = \{x = 0, y = 0\}$. Clearly, assigning any value from the interval $[5, \infty)$ to x satisfies the inequality. Therefore, the *feasible interval* for x with respect to ℓ is $[5, \infty)$.

We next describe how to construct the *sampling interval* I_s for the variable x with respect to the literal ℓ , where ℓ is the falsified literal that contains x .

- If $I_\ell = [l, \infty)$, we select contextual literals ℓ_i whose feasible intervals are of the form $(-\infty, u]$ and intersect I_ℓ with the minimal upper bound from these intervals.
- If $I_\ell = (-\infty, u]$, we select contextual literals ℓ_i whose feasible intervals are $[l, \infty)$ and intersect I_ℓ with the maximum lower bound from these intervals.

From this, we obtain the sampling interval I_s for x with respect to ℓ .

Definition 2. To satisfy a currently falsified literal ℓ with variable x , the boundary-aware move operator, denoted as $\text{bam}(\alpha, x, \ell)$, obtains a new assignment by modifying the value of x in α as an integer uniformly sampled within the sampling interval I_s determined by ℓ and $\mathcal{L}^{\text{ctx}}(x, \alpha)$.

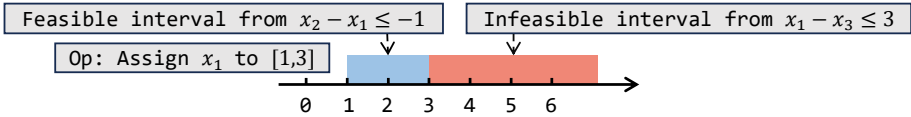


Fig. 3. During the operation $\text{bam}(\alpha, x, \ell_1)$, the infeasible interval specified by $\ell_4 : x_1 - x_3 \leq 3$ restricts the sampling interval of x_1 .

Referring to the formula given in Example 3, the current assignment is $\alpha := \{x_1 = 0, x_2 = 0, x_3 = 0\}$. When performing the operation $\text{bam}(\alpha, x_1, \ell_1)$, the initial step involves determining the feasible interval to make ℓ_1 true, resulting in $[1, \infty)$. As illustrated in Figure 3, the contextual literal ℓ_4 supplies the opposite boundary, giving $(-\infty, 3]$. Therefore, $\text{bam}(\alpha, x_1, \ell_1)$ samples a random integer for x_1 from the interval $[1, 3]$.

6 Experiments

6.1 Setup

We describe the compared state-of-the-art SMT(LIA) sampling methods, the implementation of *HighDiv*, and the benchmark suites. All experiments were conducted on a workstation equipped with with a 13th-Generation Intel Core i7-13700F CPU and 32 GB RAM.

Competitors

- **MeGASampler** [39] is a recently proposed SMT(LIA) sampling tool and is among the most advanced tools for SMT(LIA) sampling. Additionally, it implements two distinct strategies, both of which we evaluate in our experiments.
- **SMTSampler(Int)**, the integer-logic variant of SMTSampler [10], has been reported to outperform MeGASampler in terms of coverage on certain benchmarks [39], and is therefore included in our evaluation.

Implementation We implemented *HighDiv* on top of Z3 version 4.14.2⁴. To ensure a fair comparison, we used the same Z3 version (4.14.2) for MeGASampler and SMTSampler(Int).

Benchmarks To verify the effectiveness of the proposed method, we evaluated it on the same QF_LIA benchmark suite previously used to evaluate MeGASampler [39]. These benchmarks, sourced from the QF_LIA directory in SMT-LIB [2], were filtered to exclude unreasonable cases based on the following criteria: (1) marked as unsatisfiable or unknown; (2) unable to produce at least 100 samples with any technique; and (3) requiring more than one minute to solve using an SMT solver [39]. Following their methodology, they randomly selected 15 representative benchmarks from each of nine directories, yielding 345 instances, since benchmarks within the same directory tend to be similar.

6.2 Experimental Results

In this section, we first present the results of the experiment and then discuss threats to its validity.

Coverage Comparison within Fixed Time Limits To comprehensively compare state-of-the-art SMT(LIA) sampling methods, we followed the experimental setup of Peled et al. [39], setting the time limit to 900 seconds.

Table 1 presents the average coverage comparison among *HighDiv*, MeGASampler, and SMTSampler(Int) across 9 selected benchmark folders. To conserve

⁴ *HighDiv* is open-sourced at <https://github.com/laigroup/HighDiv>.

Benchmarks	Coverage with fixed time				Coverage with fixed size			
	<i>HighDiv</i>	MeGA	MeGA ^b	SMTInt	<i>HighDiv</i>	MeGA	MeGA ^b	SMTInt
CAV2009-slacked	93.15	70.15	44.77	64.24	93.16	38.03	23.56	63.71
CAV2009	76.79	44.58	69.84	55.98	76.79	34.22	40.74	52.04
convert	20.69	9.23	8.48	15.02	20.78	4.04	5.95	19.84
dillig	93.41	33.58	89.75	44.83	93.41	32.87	42.19	41.01
prime-cone	75.31	46.14	30.37	46.28	75.30	28.49	21.94	40.60
slacks	95.11	71.39	47.04	64.27	95.11	39.73	25.41	62.64
pb2010	4.45	4.41	4.39	2.92	4.73	4.68	5.70	5.09
bofill-sched-random	11.06	13.14	9.56	9.86	10.84	13.20	11.16	12.35
bofill-sched-real	10.30	11.46	9.72	8.85	10.72	11.52	10.38	11.00
Average Coverage	53.36	33.79	34.88	34.69	53.43	22.98	20.78	34.25

Table 1. Comparative results (averaged) across the benchmarks with fixed Sampling Time (900s), fixed sample size (1000), and percentage coverage.

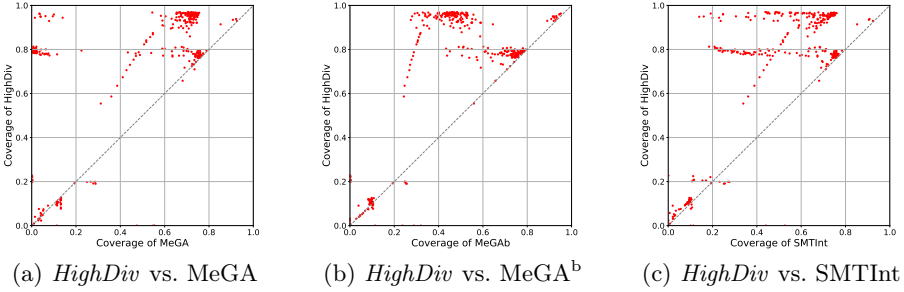


Fig. 4. Comparative Coverage Performance of *HighDiv* Against Competitors ($t = 900$ seconds).

space, only the average values for these 9 folders are shown. **MeGA** refers to the MeGASampler method based on random sampling, while **MeGA^b** denotes the blocking-based variant. **SMTInt** indicates the integer logic version of SMT-Sampler. The best coverage results are highlighted in **bold**.

As shown in Table 1, *HighDiv* achieves higher coverage than its competitors across most benchmark categories. However, for two **bofill** categories, *HighDiv* achieves lower coverage than MeGA. We attribute this result to the high proportion of equality constraints in these instances, which limits *HighDiv*'s ability to introduce diversity during the search process.

To present detailed results for each individual benchmark file, we depict scatter plots in Figure 4 to provide a benchmark-wise comparison. It can be observed that, under the same time limit, *HighDiv* yields higher coverage in most benchmarks compared to MeGASampler and SMTSampler(Int).

Coverage Comparison within Fixed Sample Size We compare the performance of *HighDiv* with its competitors using a fixed-size sample set. In practice, solutions generated via sampling are typically embedded in the test suite, and executing even a single test campaign is computationally expensive. Therefore, a fixed-size set of test cases is commonly adopted for evaluation [31].

Following prior SAT sampling studies, we set the solution set size to $k = 1000$ [31] and limited the sampling time to one hour. From Table 1, it can be seen that with $k = 1000$, *HighDiv* significantly outperforms MeGASampler and SMTSampler(Int) in terms of coverage in most benchmarks. Figure 5 shows a scatter plot comparing the coverage of *HighDiv* with its competitors, where each point represents a benchmark file.

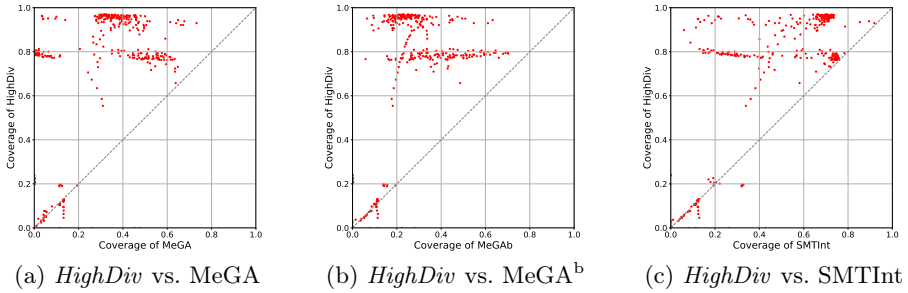


Fig. 5. Comparative Coverage Performance of *HighDiv* Against Competitors ($k = 1000$).

Effects of Algorithmic Components To analyze the effectiveness of each algorithmic component in *HighDiv*, we modify *HighDiv* to obtain seven alternative versions as follows.

- **init2zero**: In the local search phase, all integer variables are initially set to zero.
- **init2CD**: In the local search phase, all integer variables are initialized through guidance from CDCL(T).
- **init2rCD**: In the local search phase, all integer variables are initialized through reverse guidance from CDCL(T).
- **init2R**: In the local search phase, all integer variables are initialized randomly.
- **no_bam**: Removing the *boundary-aware move* operator and using the critical move operator.
- **no_cdcl**: Removing the stochastic CDCL(T) component and use only the local search component for iteration.
- **no_eqs**: Removing Z3's *solve-eqs* tactic in the pre-processing stage.

coverage	<i>HighDiv</i>	init2zero	init2CD	init2rCD	init2R	no_bam	no_cdcl	no_eqs
$t = 900s$	53.36%	47.75%	38.93%	51.09%	51.36%	33.82%	49.27%	49.71%
$k = 1000$	53.43%	47.42%	37.90%	51.15%	51.51%	33.44%	50.30%	49.86%

Table 2. Ablation study showing the effectiveness of each algorithmic component.

Table 2 presents the average coverage of each version across 9 benchmark categories. It can be observed that, under both the 900-second time limit and the 1000-sample constraint, *HighDiv* achieves higher coverage compared to the various variants. This demonstrates the effectiveness of each core algorithmic component in *HighDiv*.

Effect of Hyper-Parameter Setting Table 3 reports the average coverage of *HighDiv* across all benchmarks under fixed time and sample-size budgets, for different choices of λ and P . With our default setting ($\lambda = 50$), *HighDiv* attains the highest coverage in both experimental configurations. This matches our intuition: a smaller λ initializes the local search closer to the previous iteration’s solution, which speeds up solving at the cost of reduced diversity; a larger λ yields an initialization farther from the previous model, improving diversity but reducing solving efficiency. Moreover, coverage varies little across different values of P , suggesting that *HighDiv* is insensitive to this parameter.

coverage	λ			P				
	20	50	80	0.1	0.3	0.5	0.7	0.9
$t = 900s$	48.31%	53.36%	53.27%	53.35%	53.36%	53.36%	53.36%	53.34%
$k = 1000$	48.38%	53.43%	53.08%	53.43%	53.43%	53.43%	53.43%	53.42%

Table 3. Average coverage of sampling results by *HighDiv* under different hyperparameter settings.

6.3 Threats to Validity

The validity of our evaluation may be subject to the following potential threats:

Correctness of Implementation. To ensure the correctness of our implementation, we validated all generated samples to confirm their satisfaction of the given formulas, thereby guaranteeing the robustness and accuracy of the results.

Validity of the Experimental Setup. Different experimental configurations may pose potential threats to the validity of our evaluation. Therefore, following previous studies [39], we conducted experiments with a time limit of 900 seconds;

and, in accordance with recent recommendations [31], we set the sample size k to 1000 to mitigate this risk. The results in Tables 1 and 2 demonstrate that under both configurations, *HighDiv* significantly outperforms MeGASampler and SMTSampler(Int) in terms of diversity on most benchmarks.

Diversity Metric Bias. The definition and emphasis of “diversity” vary across application scenarios; thus, evaluation conclusions may depend on the chosen metric. To assess the applicability of our method under different measures, we further consider two widely used diversity metrics—Hamming Distance [37] and 2-wise Coverage [31]—and evaluate them under two settings: fixed sampling time and fixed sample size. The results show that, under both metrics, *HighDiv* outperforms MeGASampler and SMTSampler(Int) on most benchmarks. Detailed results are reported in the appendix of an extended version of this paper [24].

7 Related Works

The constrained sampling problem is a significant research topic in software testing [16, 18, 43]. Over the past decades, satisfiability (SAT) sampling has been extensively studied. Current SAT sampling methods include Markov-Chain Monte-Carlo (MCMC) [21, 22], heuristic local search [30, 31], universal hashing [12, 33, 34], and knowledge compilation [25, 26, 44], all of which can effectively perform SAT sampling with various balances between solving efficiency and theoretical guarantees. While SMT formula sampling can be tackled using existing SAT methods by encoding SMT formulas into SAT [45], this transformation loses the formulas’ high-level structure, which could otherwise improve sampling efficiency and diversity [10, 11, 39].

Dutra et al. introduced SMTSampler, the first sampler designed specifically for sampling SMT bit-vector theory formulas [10]. SMTSampler uses MaxSAT modulo theories (MAX-SMT) to generate initial random seeds and then obtains a set of solutions through syntactic mutation and combination. Subsequent research, GuidedSampler, allows for the setting of coverage metrics specific to certain problems and aims to optimize these metrics [11]. These methods primarily handle fixed-width bit-vector theory formulas, but this approach is unable to efficiently sample the linear integer constraints. Subsequently, Peled et al. proposed an effective sampling method for SMT(LIA) formula, named MeGASampler [39]. This method first uses an existing SMT solver to solve constraints, then generates an under-approximation formula of the original formula by adding extra constraints to the obtained solutions, and finally performs sampling based on this under-approximation formula. This is considered the state-of-the-art solution to SMT(LIA) sampling problems. However, these SMT samplers regard SMT solvers as black boxes and fail to introduce diversity into the solving process, thereby limiting the diversity of the generated solutions. Furthermore, these samplers frequently invoke MaxSMT to increase the randomness of the initial seeds, but using MaxSMT is very expensive, thus creating substantial overhead.

8 Conclusion

Previous SMT sampling algorithms often struggle to generate diverse samples as they treat the SMT solver as a black box and rely on extending a single model, limiting the coverage of the solution space. To overcome this, we propose *HighDiv*, the first iterative SMT sampling framework that integrates CDCL(T) and local search in a bidirectional guided manner. In the local search phase, *HighDiv* introduces an *Constraint-Partitioned Variable Initialization* strategy and the *boundary-aware move* operator, effectively balancing sample diversity and search efficiency. Experimental results show that *HighDiv* produces significantly more diverse samples than MeGASampler and SMTSampler(Int).

It is important to note that *HighDiv* is orthogonal to existing SMT sampling algorithms such as MeGASampler and SMTSampler. By generating initial models with *HighDiv* and then applying model-guided approximation or combinatorial mutation techniques, overall sampling efficiency can be greatly improved. In future work, we plan to extend *HighDiv* to support additional theories, such as Linear Real Arithmetic and Quantifier-Free Bit-Vectors, and explore integrating model-guided approximation and combinatorial mutation strategies to further enhance its scalability.

Acknowledgements

We thank the anonymous reviewers for their helpful feedback and Zizhao Han for artifact testing. This work was supported in part by National Key Research and Development Program of China [2023YFB3307500], Jilin Provincial Natural Science Foundation [20240101378JC], Jilin Provincial Education Department Research Project [JJKH20241286KJ], National Natural Science Foundation of China [62522201, 62202025, U22A2098, and 62172185], Young Elite Scientist Sponsorship Program by CAST [YESS20230566], Beijing Natural Science Foundation [L241050], CCF-Huawei Populus Grove Fund [CCF-HuaweiFM2024005], and Fundamental Research Fund Project of Beihang University.

References

1. Barbosa, H., Barrett, C.W., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In: Fisman, D., Rosu, G. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022. Lecture Notes in Computer Science, vol. 13243, pp. 415–442 (2022)
2. Barrett, C., Fontaine, P., Tinelli, C.: The Satisfiability Modulo Theories Library (SMT-LIB). www.SMT-LIB.org (2016)
3. Barrett, C.W., Sebastiani, R., Seshia, S.A., Tinelli, C.: Satisfiability modulo theories. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability - Second Edition, Frontiers in Artificial Intelligence and Applications, vol. 336, pp. 1267–1329 (2021)

4. Cadar, C., Dunbar, D., Engler, D.R.: KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In: Draves, R., van Renesse, R. (eds.) 8th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2008. pp. 209–224 (2008)
5. Cai, S., Li, B., Zhang, X.: Local search for SMT on linear integer arithmetic. In: Shoham, S., Vizel, Y. (eds.) Computer Aided Verification - 34th International Conference, CAV 2022. Lecture Notes in Computer Science, vol. 13372, pp. 227–248 (2022)
6. Cai, S., Su, K.: Local search for boolean satisfiability with configuration checking and subscore. *Artificial Intelligence*
7. Carrasco, M., Cadar, C., Donaldson, A.: Scalable smt sampling for floating-point formulas via coverage-guided fuzzing. In: IEEE International Conference on Software Testing, Verification, and Validation (ICST 2025) (2025)
8. Cimatti, A., Griggio, A., Schaafsma, B.J., Sebastiani, R.: The mathsat5 SMT solver. In: Piterman, N., Smolka, S.A. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 19th International Conference, TACAS 2013. Lecture Notes in Computer Science, vol. 7795, pp. 93–107 (2013)
9. Codish, M., Fekete, Y., Fuhs, C., Giesl, J., Waldmann, J.: Exotic semi-ring constraints. In: Fontaine, P., Goel, A. (eds.) 10th International Workshop on Satisfiability Modulo Theories, SMT 2012. EPIc Series in Computing, vol. 20, pp. 88–97 (2012)
10. Dutra, R., Bachrach, J., Sen, K.: Smtsamplers: efficient stimulus generation from complex SMT constraints. In: Bahar, I. (ed.) Proceedings of the International Conference on Computer-Aided Design, ICCAD 2018. p. 30 (2018)
11. Dutra, R., Bachrach, J., Sen, K.: GUIDEDSAMPLER: coverage-guided sampling of SMT solutions. In: Barrett, C.W., Yang, J. (eds.) 2019 Formal Methods in Computer Aided Design, FMCAD 2019. pp. 203–211 (2019)
12. Ermon, S., Gomes, C.P., Sabharwal, A., Selman, B.: Embed and project: Discrete sampling with universal hashing. In: Burges, C.J.C., Bottou, L., Ghahramani, Z., Weinberger, K.Q. (eds.) Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. pp. 2085–2093 (2013)
13. Fröhlich, A., Biere, A., Wintersteiger, C.M., Hamadi, Y.: Stochastic local search for satisfiability modulo theories. In: Bonet, B., Koenig, S. (eds.) Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence. pp. 1136–1143 (2015)
14. Ganzinger, H., Hagen, G., Nieuwenhuis, R., Oliveras, A., Tinelli, C.: DPLL(T): fast decision procedures. In: Alur, R., Peled, D.A. (eds.) Computer Aided Verification, 16th International Conference, CAV 2004. Lecture Notes in Computer Science, vol. 3114, pp. 175–188 (2004)
15. Gavrilenco, N., de León, H.P., Furbach, F., Heljanko, K., Meyer, R.: BMC for weak memory models: Relation analysis for compact SMT encodings. In: Dillig, I., Tasiran, S. (eds.) Computer Aided Verification - 31st International Conference, CAV 2019. Lecture Notes in Computer Science, vol. 11561, pp. 355–365 (2019)
16. Godefroid, P., Klarlund, N., Sen, K.: DART: directed automated random testing. In: Sarkar, V., Hall, M.W. (eds.) Proceedings of the ACM SIGPLAN 2005 Conference on Programming Language Design and Implementation. pp. 213–223 (2005)
17. Golia, P., Soos, M., Chakraborty, S., Meel, K.S.: Designing samplers is easy: The boon of testers. In: Formal Methods in Computer Aided Design, FMCAD 2021. pp. 222–230 (2021)
18. Holler, C., Herzig, K., Zeller, A.: Fuzzing with code fragments. In: Kohn, T. (ed.) Proceedings of the 21th USENIX Security Symposium. pp. 445–458 (2012)

19. Huang, H., Yao, P., Wu, R., Shi, Q., Zhang, C.: Pangolin: Incremental hybrid fuzzing with polyhedral path abstraction. In: 2020 IEEE Symposium on Security and Privacy, SP 2020. pp. 1613–1627 (2020)
20. Jiang, L., Yuan, H., Wu, M., Zhang, L., Zhang, Y.: Evaluating and improving hybrid fuzzing. In: 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023. pp. 410–422 (2023)
21. Kitchen, N.: Markov Chain Monte Carlo Stimulus Generation for Constrained Random Simulation. Ph.D. thesis, University of California, Berkeley, USA (2010)
22. Kitchen, N., Kuehlmann, A.: Stimulus generation for constrained random simulation. In: Gielen, G.G.E. (ed.) 2007 International Conference on Computer-Aided Design, ICCAD 2007. pp. 258–265 (2007)
23. Kroening, D., Strichman, O.: Decision Procedures - An Algorithmic Point of View, Second Edition. Texts in Theoretical Computer Science. An EATCS Series, Springer (2016)
24. Lai, Y., Li, J., Luo, C.: SMT(LIA) sampling with high diversity. arXiv preprint arXiv:2503.04782 (2025)
25. Lai, Y., Liu, D., Yin, M.: New canonical representations by augmenting obdds with conjunctive decomposition. vol. 58, pp. 453–521 (2017)
26. Lai, Y., Meel, K.S., Yap, R.H.: Panini: an efficient and flexible knowledge compiler. In: International Conference on Computer Aided Verification. pp. 92–105. Springer (2025)
27. Liu, C., Liu, G., Luo, C., Cai, S., Lei, Z., Zhang, W., Chu, Y., Zhang, G.: Optimizing local search-based partial maxsat solving via initial assignment prediction. *Sci. China Inf. Sci.* **68**(2) (2025)
28. Liu, D., Ernst, G., Murray, T., Rubinstein, B.I.P.: LEGION: best-first concolic testing. In: 35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020. pp. 54–65 (2020)
29. Lopes, N.P., Monteiro, J.: Automatic equivalence checking of programs with uninterpreted functions and integer arithmetic. *Int. J. Softw. Tools Technol. Transf.* **18**(4), 359–374 (2016)
30. Luo, C., Song, J., Zhao, Q., Sun, B., Chen, J., Zhang, H., Lin, J., Hu, C.: Solving the t -wise coverage maximum problem via effective and efficient local search-based sampling. *ACM Trans. Softw. Eng. Methodol.* **34**(1), 13:1–13:64 (2025)
31. Luo, C., Sun, B., Qiao, B., Chen, J., Zhang, H., Lin, J., Lin, Q., Zhang, D.: Ls-sampling: an effective local search based sampling approach for achieving high t -wise coverage. In: Spinellis, D., Gousios, G., Chechik, M., Penta, M.D. (eds.) ESEC/FSE ’21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 1081–1092 (2021)
32. McCarthy, J.: Towards a mathematical science of computation. In: Information Processing, Proceedings of the 2nd IFIP Congress 1962, pp. 21–28 (1962)
33. Meel, K.S.: Sampling techniques for boolean satisfiability. *CoRR* **abs/1404.6682** (2014)
34. Meel, K.S., Vardi, M.Y., Chakraborty, S., Fremont, D.J., Seshia, S.A., Fried, D., Ivrii, A., Malik, S.: Constrained sampling and counting: Universal hashing meets SAT solving. In: Darwiche, A. (ed.) Beyond NP, Papers from the 2016 AAAI Workshop. AAAI Technical Report, vol. WS-16-05 (2016)
35. Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S.: Chaff: Engineering an efficient sat solver. In: Proceedings of the 38th annual Design Automation Conference. pp. 530–535 (2001)

36. de Moura, L.M., Bjørner, N.S.: Z3: an efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*, 14th International Conference, TACAS 2008. *Lecture Notes in Computer Science*, vol. 4963, pp. 337–340 (2008)
37. Nadel, A.: Generating diverse solutions in sat. In: *International Conference on Theory and Applications of Satisfiability Testing*. pp. 287–301. Springer (2011)
38. Naveh, Y., Rimón, M., Jaeger, I., Katz, Y., Vinov, M., Marcus, E., Shurek, G.: Constraint-based random stimuli generation for hardware verification pp. 1720–1727 (2006)
39. Peled, M., Rothenberg, B., Itzhaky, S.: SMT sampling via model-guided approximation. In: Chechik, M., Katoen, J., Leucker, M. (eds.) *Formal Methods - 25th International Symposium, FM 2023*. *Lecture Notes in Computer Science*, vol. 14000, pp. 74–91 (2023)
40. Peleska, J., Vorobev, E., Lapschies, F.: Automated test case generation with smt-solving and abstract interpretation. In: Bobaru, M.G., Havelund, K., Holzmann, G.J., Joshi, R. (eds.) *NASA Formal Methods - Third International Symposium, NFM 2011*. *Lecture Notes in Computer Science*, vol. 6617, pp. 298–312 (2011)
41. Pipatsrisawat, K., Darwiche, A.: A lightweight component caching scheme for satisfiability solvers. In: *International conference on theory and applications of satisfiability testing*. pp. 294–299. Springer (2007)
42. Poeplau, S., Francillon, A.: Symbolic execution with symcc: Don’t interpret, compile! In: Capkun, S., Roesner, F. (eds.) *29th USENIX Security Symposium, USENIX Security 2020*. pp. 181–198 (2020)
43. Sen, K., Marinov, D., Agha, G.: CUTE: a concolic unit testing engine for C. In: Wermelinger, M., Gall, H.C. (eds.) *Proceedings of the 10th European Software Engineering Conference held jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2005. pp. 263–272 (2005)
44. Sharma, S., Gupta, R., Roy, S., Meel, K.S.: Knowledge compilation meets uniform sampling. In: Barthe, G., Sutcliffe, G., Veanes, M. (eds.) *LPAR-22. 22nd International Conference on Logic for Programming, Artificial Intelligence and Reasoning*. *EPiC Series in Computing*, vol. 57, pp. 620–636 (2018)
45. Shaw, A., Meel, K.S.: CSB: A counting and sampling tool for bit-vectors. In: Reger, G., Zohar, Y. (eds.) *Proceedings of the 22nd International Workshop on Satisfiability Modulo Theories co-located with the 36th International Conference on Computer Aided Verification (CAV 2024)*. *CEUR Workshop Proceedings*, vol. 3725, pp. 36–43 (2024)
46. Thornton, J., Pham, D.N., Bain, S., Ferreira Jr, V.: Additive versus multiplicative clause weighting for sat. In: *AAAI*. vol. 4, pp. 191–196 (2004)
47. Zhang, X., Li, B., Cai, S.: Deep combination of CDCL(T) and local search for satisfiability modulo non-linear integer arithmetic theory. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024*. pp. 125:1–125:13 (2024)
48. Zhang, Y., Chen, Z., Shuai, Z., Zhang, T., Li, K., Wang, J.: Multiplex symbolic execution: Exploring multiple paths by solving once. In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. pp. 846–857 (2020)

Open Access. This chapter is licensed under the terms of the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (<http://creativecommons.org/licenses/by-nc-nd/4.0/>), which permits any noncommercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if you modified the licensed material. You do not have permission under this license to share adapted material derived from this chapter or parts of it.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

